

InterfaceHive Backend Architecture

A robust Django-based REST API with real-time capabilities for a contribution marketplace platform



Django 5.0



PostgreSQL 16



Redis 7



Celery 5.3



Technology Stack & Architecture Overview

Core Technologies

</> Core Framework

Python 3.11+

Django 5.0

DRF 3.14

🗄️ Data & Processing

PostgreSQL 16

Redis 7

Celery 5.3

🏗️ Modular Structure

Users App

Projects App

Contributions

Credits System

Architecture Patterns

📁 Layered Architecture

</> API Layer Views & Endpoints

↔️ Serialization DRF Serializers

⚙️ Business Logic Services

🗄️ Data Access Models & ORM

Key Principles:

🧩 Service Layer Pattern

📡 Signal-Driven Events

📁 Separation of Concerns

🏗️ Modular Organization

Core Applications & Features

Users App

JWT Authentication
Short-lived access tokens with refresh

Email Verification
Secure account activation

Custom User Model
UUID primary key, unique email

POST /api/v1/auth/register/

POST /api/v1/auth/login/

Projects App

Full-text Search
GIN indexes for title/description

Project Workflow
CRUD with closure capability

Tagging System
ArrayField for categorization

GET /api/v1/projects/

POST /api/v1/projects/

Contributions

1. Submission
Contributor submits work

2. Review
Host evaluates

3. Decision
Accept/Decline

POST /api/v1/contributions/

Immutable Credit System



Append-Only Ledger
Cannot be modified



Real-time Balance
Calculated on demand



Reversible
Via reversal entries



Audit Trail
Complete transaction history

1 Credit = 1 Point

Security, API Design & Real-time Features

Security

JWT Authentication

Access (15 min) & Refresh (7 days) tokens

Permission Classes

IsAuthenticated, IsOwnerOrReadOnly

Input Validation

DRF serializers & DB constraints

CORS Configuration

Secure cross-origin sharing

Security Flow:

1. CORS → 2. JWT → 3. Permissions → 4. Validation

API Design

RESTful Conventions

- Resource-based URLs (/projects/)
- HTTP verbs (GET, POST, PATCH, DELETE)
- Consistent response format

Response Format

```
{
  "success": true,
  "data": { ... },
  "message": "OK"
}
```

Database Optimization

- GIN indexes for full-text search
- select_related() for ForeignKey
- Connection pooling (pgBouncer)

Real-time Chat

WebSocket Architecture



Client



Daphne



Channels



Redis

Key Features

- Project-scoped conversations
- Message persistence in PostgreSQL
- Redis-backed channel layer
- Real-time message delivery

Technical Stack

 Django Channels 4.0

 Daphne ASGI

Integrated Architecture

Security → API → Real-time → Database



JWT + Permissions protect all endpoints



RESTful API with consistent responses



WebSocket chat with message persistence



Optimized queries with strategic indexing

Deployment, Scalability & Benefits

Deployment Architecture



- ✓ Stateless API with JWT authentication
- ✓ Multiple Gunicorn workers for concurrency

Scalability Features

Horizontal Scaling

- Stateless API design
- Load balancer distribution
- Multiple worker processes

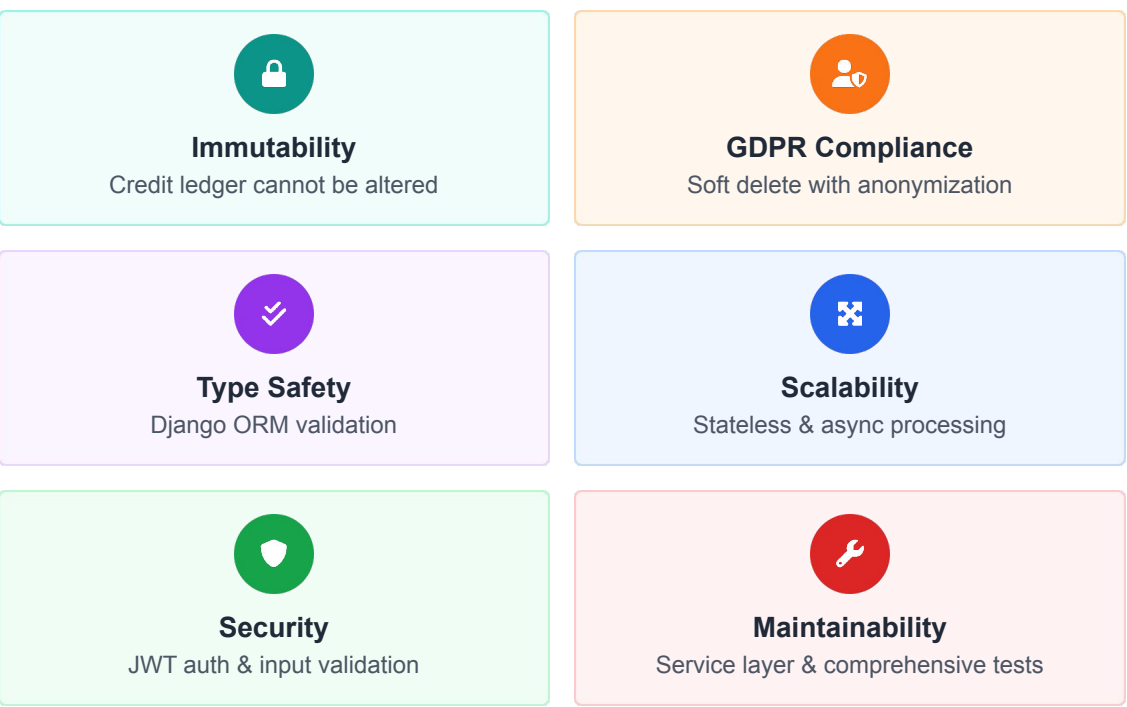
Database Optimization

- Read replicas for analytics
- Connection pooling (pgBouncer)
- Strategic indexing (GIN, B-tree)

Caching Strategy

Redis Sessions Query Cache CDN Static

Key Benefits



Architecture Summary

InterfaceHive's backend provides a robust, scalable foundation for the contribution marketplace with emphasis on security, performance, and maintainability.